

# B1\_Data\_Audit\_Feature\_Engineering

June 14, 2026

## 1 Credit Risk Data Audit and Feature Engineering

---

|                 |                                                                                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Notebook</b> | B1_Data_Audit_Feature_Engineering.ipynb                                                                                                                             |
| <b>Author</b>   | N L N Sai Krishna Akula                                                                                                                                             |
| <b>Course</b>   | AAI 500 - Applied Statistics & AI                                                                                                                                   |
| <b>Dataset</b>  | Credit Risk Modeling Dataset                                                                                                                                        |
| <b>Date</b>     | June 3, 2026                                                                                                                                                        |
| <b>GitHub</b>   | <a href="https://github.com/diaeshantony-usd/applied-statistics-and-ai-team-project">https://github.com/diaeshantony-usd/applied-statistics-and-ai-team-project</a> |

---

This notebook loads the Phase 1 cleaned credit-risk dataset, audits it, creates feature-engineered columns, and saves the Phase 2 modeling dataset for the next notebook.

```
[110]: # Core data libraries
import warnings
from pathlib import Path

import numpy as np
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# VIF is used to check multicollinearity between predictor variables.
from statsmodels.stats.outliers_influence import variance_inflation_factor

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 100)

PHASE1_DATA_PATH = "../data/processed/phase1/credit_risk_cleaned.csv"
PHASE2_DATA_PATH = "../data/processed/phase2/credit_risk_cleaned.csv"

print("Libraries loaded successfully.")
print("Phase 1 dataset configured:", PHASE1_DATA_PATH)
print("Phase 2 dataset configured:", PHASE2_DATA_PATH)
```

Libraries loaded successfully.

Phase 1 dataset configured: ../data/processed/phase1/credit\_risk\_cleaned.csv

Phase 2 dataset configured: ../data/processed/phase2/credit\_risk\_cleaned.csv

---

### 1.0.1 1.3 Load and Audit the Phase 1 Cleaned Dataset

The modeling workflow starts from the Phase 1 cleaned dataset saved at data/processed/phase1/credit\_risk\_cleaned.csv. The audit is separated into small outputs so each check is easy to explain in a final project presentation.

The first check confirms that the cleaned file loads correctly and shows the dataset size.

```
[111]: df = pd.read_csv(PHASE1_DATA_PATH)

# Print a short load confirmation without exposing any local full file path.
print("Dataset loaded successfully.")
print("Shape:", df.shape)
```

Dataset loaded successfully.

Shape: (29567, 12)

The first five rows provide a quick visual check of the columns and sample values.

```
[112]: # Display sample records to verify the dataset structure before modeling.
df.head()
```

```
[112]:
```

|   | person_age | person_income | person_home_ownership | person_emp_length | \ |
|---|------------|---------------|-----------------------|-------------------|---|
| 0 | 21         | 9600          | OWN                   | 5.0               |   |
| 1 | 25         | 9600          | MORTGAGE              | 1.0               |   |
| 2 | 23         | 65500         | RENT                  | 4.0               |   |
| 3 | 24         | 54400         | RENT                  | 8.0               |   |
| 4 | 21         | 9900          | OWN                   | 2.0               |   |

|   | loan_intent | loan_grade | loan_amnt | loan_int_rate | loan_status | \ |
|---|-------------|------------|-----------|---------------|-------------|---|
| 0 | EDUCATION   | B          | 1000      | 11.14         | 0           |   |
| 1 | MEDICAL     | C          | 5500      | 12.87         | 1           |   |
| 2 | MEDICAL     | C          | 35000     | 15.23         | 1           |   |
| 3 | MEDICAL     | C          | 35000     | 14.27         | 1           |   |
| 4 | VENTURE     | A          | 2500      | 7.14          | 1           |   |

|   | loan_percent_income | cb_person_default_on_file | cb_person_cred_hist_length |
|---|---------------------|---------------------------|----------------------------|
| 0 | 0.10                | N                         | 2                          |
| 1 | 0.57                | N                         | 3                          |
| 2 | 0.53                | N                         | 2                          |
| 3 | 0.55                | Y                         | 4                          |
| 4 | 0.25                | N                         | 2                          |

The preview confirms that borrower details, loan details, and credit history fields are available for modeling. This check helps ensure that later feature engineering uses the expected columns.

```
[113]: # Collect categorical columns and list their levels before encoding.
categorical_cols_initial = df.select_dtypes(include=["object", "category"]).
    ↪columns.tolist()
categorical_summary = {
    col: sorted(df[col].dropna().astype(str).unique().tolist())
    for col in categorical_cols_initial
}
pd.DataFrame({
    "column": list(categorical_summary.keys()),
    "levels": ["", ".join(vals) for vals in categorical_summary.values()],
})
```

```
[113]:          column \
0      person_home_ownership
1          loan_intent
2          loan_grade
3  cb_person_default_on_file

          levels
0      MORTGAGE, OTHER, OWN, RENT
1  DEBTCONSOLIDATION, EDUCATION, HOMEIMPROVEMENT,...
2          A, B, C, D, E, F, G
3          N, Y
```

Numeric summary statistics show the scale and range of each numeric field before feature engineering.

```
[114]: # Review numeric ranges and distributions before feature engineering.
df.describe().T.round(3)
```

```
[114]:
```

|                            | count   | mean      | std       | min     | 25%      | \ |
|----------------------------|---------|-----------|-----------|---------|----------|---|
| person_age                 | 29567.0 | 26.726    | 4.507     | 20.00   | 23.00    |   |
| person_income              | 29567.0 | 58588.391 | 27477.043 | 4080.00 | 37863.00 |   |
| person_emp_length          | 29567.0 | 4.629     | 3.769     | 0.00    | 2.00     |   |
| loan_amnt                  | 29567.0 | 9278.747  | 6024.906  | 500.00  | 5000.00  |   |
| loan_int_rate              | 29567.0 | 11.000    | 3.078     | 5.42    | 8.49     |   |
| loan_status                | 29567.0 | 0.224     | 0.417     | 0.00    | 0.00     |   |
| loan_percent_income        | 29567.0 | 0.175     | 0.107     | 0.01    | 0.09     |   |
| cb_person_cred_hist_length | 29567.0 | 5.269     | 3.315     | 2.00    | 3.00     |   |

|                     | 50%      | 75%      | max       |
|---------------------|----------|----------|-----------|
| person_age          | 25.00    | 29.00    | 40.00     |
| person_income       | 54000.00 | 75000.00 | 140004.00 |
| person_emp_length   | 4.00     | 7.00     | 25.00     |
| loan_amnt           | 8000.00  | 12000.00 | 35000.00  |
| loan_int_rate       | 10.99    | 13.11    | 23.22     |
| loan_status         | 0.00     | 0.00     | 1.00      |
| loan_percent_income | 0.15     | 0.23     | 0.83      |

|                            |      |      |       |
|----------------------------|------|------|-------|
| cb_person_cred_hist_length | 4.00 | 7.00 | 17.00 |
|----------------------------|------|------|-------|

---

## 1.1 2. Feature Engineering and Exploratory Analysis

### 1.1.1 2.1 Feature Engineering

This step creates additional columns that make borrower affordability, loan burden, and credit history easier to analyze. The new columns describe income level, loan burden, age group, and past default behavior.

Log transformations reduce the influence of very large financial values. Ratio features compare borrower capacity with requested loan size. Grouped features such as `income_group`, `loan_amount_group`, `loan_percent_income_group`, and `age_group` make default patterns easier to explain.

```
[115]: def add_credit_features(data):
    # Copy the input so the original cleaned dataset is not modified directly.
    out = data.copy()

    # Log transforms reduce the influence of very large income and loan values.
    out["log_person_income"] = np.log1p(out["person_income"])
    out["log_loan_amnt"] = np.log1p(out["loan_amnt"])

    # Ratio features describe affordability and borrower history in relative
    # terms.
    out["income_to_loan_ratio"] = out["person_income"] / out["loan_amnt"].
    clip(lower=1)
    out["employment_to_age_ratio"] = out["person_emp_length"] /
    out["person_age"].clip(lower=1)
    out["credit_history_to_age_ratio"] = out["cb_person_cred_hist_length"] /
    out["person_age"].clip(lower=1)
    out["default_on_file_binary"] = (out["cb_person_default_on_file"] == "Y").
    astype(int)

    # Numeric grade is useful for analysis while model preprocessing still
    # handles grade encoding.
    grade_map = {"A": 0, "B": 1, "C": 2, "D": 3, "E": 4, "F": 5, "G": 6}
    out["loan_grade_numeric"] = out["loan_grade"].map(grade_map)

    # Grouped features make borrower and loan risk patterns easier to explain.
    out["income_group"] = pd.cut(
        out["person_income"],
        bins=[0, 25000, 50000, 75000, 100000, np.inf],
        labels=["Low", "Lower-Middle", "Middle", "Upper-Middle", "High"],
        include_lowest=True,
    )
```

```

out["loan_amount_group"] = pd.cut(
    out["loan_amnt"],
    bins=[0, 5000, 10000, 20000, np.inf],
    labels=["Small", "Medium", "Large", "Very Large"],
    include_lowest=True,
)
out["loan_percent_income_group"] = pd.cut(
    out["loan_percent_income"],
    bins=[0, 0.10, 0.20, 0.35, np.inf],
    labels=["Low Burden", "Moderate Burden", "High Burden", "Very High_
↳Burden"],
    include_lowest=True,
)
out["age_group"] = pd.cut(
    out["person_age"],
    bins=[0, 25, 35, np.inf],
    labels=["Young", "Adult", "Experienced Adult"],
    include_lowest=True,
)
# Return the feature-engineered dataframe for all later modeling steps.
return out

```

```

[116]: df_model = add_credit_features(df)

# Print the new shape after engineered columns are added.
print("Feature-engineered dataset shape:", df_model.shape)

```

Feature-engineered dataset shape: (29567, 23)

```

[117]: # Preview engineered columns to confirm the feature creation step.
df_model.head()

```

```

[117]:   person_age  person_income  person_home_ownership  person_emp_length \
0          21          9600                OWN                5.0
1          25          9600             MORTGAGE                1.0
2          23         65500                RENT                4.0
3          24         54400                RENT                8.0
4          21          9900                OWN                2.0

   loan_intent  loan_grade  loan_amnt  loan_int_rate  loan_status \
0  EDUCATION          B      1000        11.14            0
1   MEDICAL          C      5500        12.87            1
2   MEDICAL          C     35000        15.23            1
3   MEDICAL          C     35000        14.27            1
4   VENTURE          A      2500         7.14            1

   loan_percent_income  cb_person_default_on_file  cb_person_cred_hist_length \
0              0.10                                N                        2

```

|   |      |   |   |
|---|------|---|---|
| 1 | 0.57 | N | 3 |
| 2 | 0.53 | N | 2 |
| 3 | 0.55 | Y | 4 |
| 4 | 0.25 | N | 2 |

|   | log_person_income | log_loan_amnt | income_to_loan_ratio \ |
|---|-------------------|---------------|------------------------|
| 0 | 9.169623          | 6.908755      | 9.600000               |
| 1 | 9.169623          | 8.612685      | 1.745455               |
| 2 | 11.089821         | 10.463132     | 1.871429               |
| 3 | 10.904138         | 10.463132     | 1.554286               |
| 4 | 9.200391          | 7.824446      | 3.960000               |

|   | employment_to_age_ratio | credit_history_to_age_ratio \ |
|---|-------------------------|-------------------------------|
| 0 | 0.238095                | 0.095238                      |
| 1 | 0.040000                | 0.120000                      |
| 2 | 0.173913                | 0.086957                      |
| 3 | 0.333333                | 0.166667                      |
| 4 | 0.095238                | 0.095238                      |

|   | default_on_file_binary | loan_grade_numeric | income_group | loan_amount_group \ |
|---|------------------------|--------------------|--------------|---------------------|
| 0 | 0                      | 1                  | Low          | Small               |
| 1 | 0                      | 2                  | Low          | Medium              |
| 2 | 0                      | 2                  | Middle       | Very Large          |
| 3 | 1                      | 2                  | Middle       | Very Large          |
| 4 | 0                      | 0                  | Low          | Small               |

|   | loan_percent_income_group | age_group |
|---|---------------------------|-----------|
| 0 | Low Burden                | Young     |
| 1 | Very High Burden          | Young     |
| 2 | Very High Burden          | Young     |
| 3 | Very High Burden          | Young     |
| 4 | High Burden               | Young     |

### 1.1.2 2.2 Grouped Feature Default Rate EDA

Grouped features help explain whether risk changes across borrower and loan categories. The outputs below show default rate separately for income group, loan amount group, and loan burden group.

```
[118]: def summarize_default_rate_by_group(data, group_col):
        # Group by the selected category and calculate default rate plus loan-size
        ↪context.
        summary = (
            data.groupby(group_col, observed=False)
            .agg(
                loan_count=("loan_status", "size"),
```

```

        default_rate=("loan_status", "mean"),
        avg_loan_amount=("loan_amnt", "mean"),
        avg_loan_percent_income=("loan_percent_income", "mean"),
    )
    .reset_index()
)
# Return a compact table that can be displayed or plotted.
return summary

```

The income group table shows whether lower or higher income bands have different default rates.

```

[119]: income_group_summary = summarize_default_rate_by_group(df_model, "income_group")

# Display default rate by income band.
income_group_summary.style.format({
    "default_rate": "{:.2%}",
    "avg_loan_amount": "{:.2f}",
    "avg_loan_percent_income": "{:.2%}",
})

```

```
[119]: <pandas.io.formats.style.Styler at 0x210c0920650>
```

This table compares default behavior across income groups. Higher default rates in a group suggest that income level is related to credit risk and should remain part of the modeling dataset.

The loan amount group table checks whether larger requested loans behave differently from smaller loans.

```

[120]: loan_amount_group_summary = summarize_default_rate_by_group(df_model, ↵
    ↵"loan_amount_group")

# Display default rate by loan amount band.
loan_amount_group_summary.style.format({
    "default_rate": "{:.2%}",
    "avg_loan_amount": "{:.2f}",
    "avg_loan_percent_income": "{:.2%}",
})

```

```
[120]: <pandas.io.formats.style.Styler at 0x210bf313c80>
```

This table compares default behavior across loan amount groups. It helps show whether larger requested loans are associated with different repayment outcomes.

The loan percent income group table focuses on loan burden, which is usually more informative than loan amount alone.

```

[121]: loan_percent_income_group_summary = summarize_default_rate_by_group(df_model, ↵
    ↵"loan_percent_income_group")

# Display default rate by loan burden band.

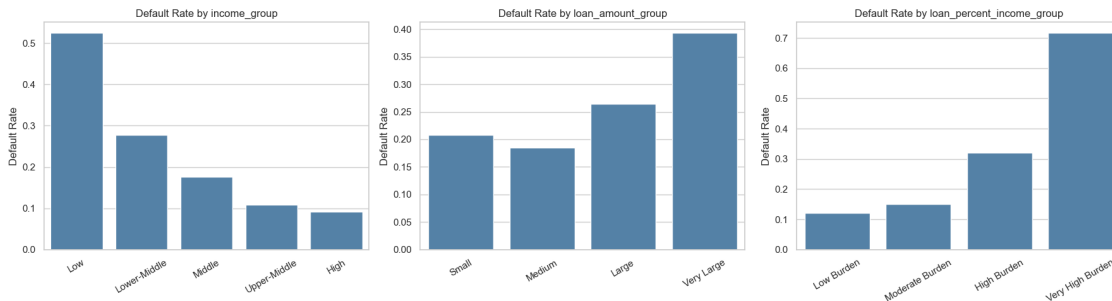
```

```
loan_percent_income_group_summary.style.format({
    "default_rate": "{:.2%}",
    "avg_loan_amount": "{:.2f}",
    "avg_loan_percent_income": "{:.2%}",
})
```

[121]: <pandas.io.formats.style.Styler at 0x210bf311280>

```
[122]: # Store grouped summaries so the plots can be generated consistently.
group_feature_summaries = {
    "income_group": income_group_summary,
    "loan_amount_group": loan_amount_group_summary,
    "loan_percent_income_group": loan_percent_income_group_summary,
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, group_name in zip(axes, group_feature_summaries.keys()):
    summary = group_feature_summaries[group_name]
    sns.barplot(data=summary, x=group_name, y="default_rate",
        color="steelblue", ax=ax)
    ax.set_title(f"Default Rate by {group_name}")
    ax.set_xlabel("")
    ax.set_ylabel("Default Rate")
    ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()
```



### 1.1.3 2.3 Credit Grade and Interest Rate Analysis

The main classification model uses an issued-loan framing, so `loan_grade` and `loan_int_rate` are included because those values are known after underwriting. This section checks how default rate and interest rate change across grades.

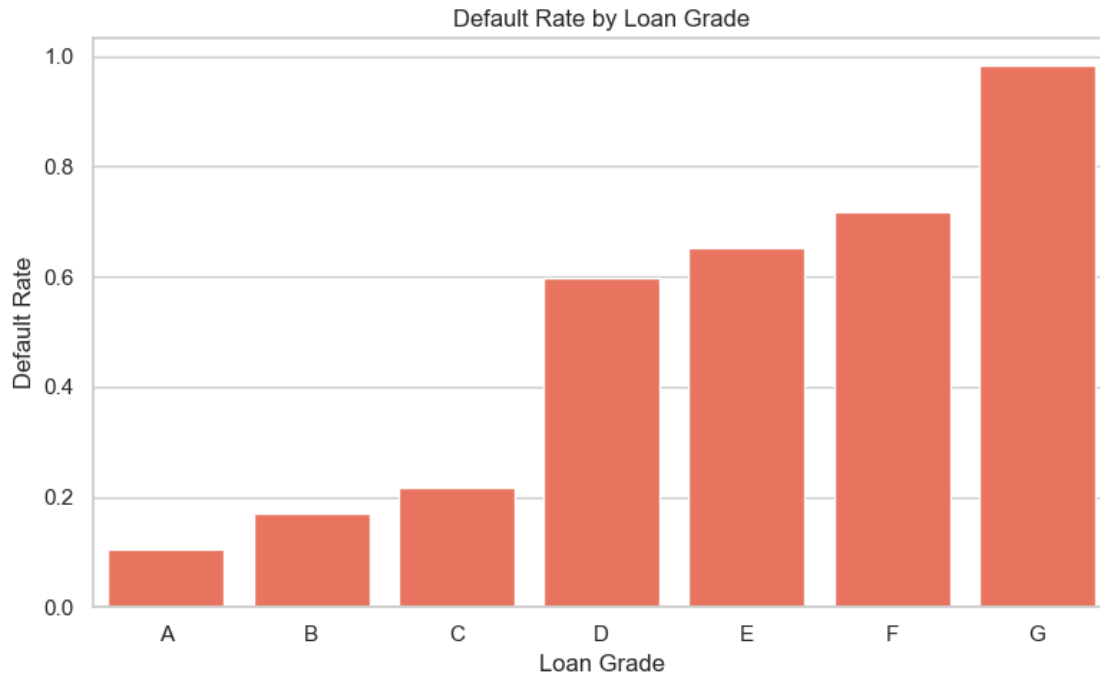


```
[123]: # Summarize default behavior and interest rate patterns by loan grade.
grade_summary = (
    df_model.groupby("loan_grade")
    .agg(
        loan_count=("loan_status", "size"),
        default_rate=("loan_status", "mean"),
        avg_interest_rate=("loan_int_rate", "mean"),
        avg_loan_percent_income=("loan_percent_income", "mean"),
    )
    .reset_index()
)
grade_summary.style.format({
    "default_rate": "{:.2%}",
    "avg_interest_rate": "{:.2f}",
    "avg_loan_percent_income": "{:.2%}",
})
```

```
[123]: <pandas.io.formats.style.Styler at 0x210bcf12930>
```

The grade and interest-rate summary shows how lender-assigned grades relate to default rates and pricing. This supports including grade and rate in the issued-loan modeling view.

```
[124]: # Plot a focused relationship so grade-level risk is easier to read.
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(data=grade_summary, x="loan_grade", y="default_rate",
    color="tomato", ax=ax)
ax.set_title("Default Rate by Loan Grade")
ax.set_xlabel("Loan Grade")
ax.set_ylabel("Default Rate")
plt.tight_layout()
plt.show()
```



```
[125]: def summarize_default_probability(data, group_cols):
        # Empirical conditional probabilities
        summary = (
            data.groupby(group_cols, observed=False)
            .agg(
                loan_count=("loan_status", "size"),
                default_probability=("loan_status", "mean"),
            )
            .reset_index()
            .sort_values("default_probability", ascending=False)
        )
        return summary

bn_probability_by_burden = summarize_default_probability(
    df_model, ["loan_percent_income_group", "cb_person_default_on_file"]
)

# Display conditional default probabilities by loan burden and prior default_
↳ flag.
bn_probability_by_burden.style.format({"default_probability": "{:.2%}"})
```

```
[125]: <pandas.io.formats.style.Styler at 0x210bf274cb0>
```

### 1.1.4 2.4 Multicollinearity Check

Multicollinearity means two or more predictor columns carry very similar information. For example, loan amount and log loan amount can move together because both describe loan size.

VIF stands for Variance Inflation Factor. It requires numeric input columns, so categorical columns are temporarily one-hot encoded only for this diagnostic check. Names such as `loan_grade_D` or `income_group_Middle` are not original saved dataset columns. They are temporary encoded columns created so VIF can be calculated.

This is similar to the B2 modeling pipeline, where text categories are converted into numeric columns before model training. The difference is that B1 uses the encoded columns only for feature-quality checking, while B2 uses encoded columns for actual model training.

```
[126]: def interpret_vif(vif_value):
    # Convert a numeric VIF value into a simple project-level label.
    if np.isinf(vif_value) or vif_value >= 10:
        return "High"
    if vif_value >= 5:
        return "Moderate"
    return "Low"

def encode_features_for_vif(data, feature_cols):
    # Keep only requested features that exist in the dataframe.
    available_features = [col for col in feature_cols if col in data.columns]
    vif_input = data[available_features].copy()

    # One-hot encode categorical columns temporarily because VIF requires
    # numeric input.
    vif_encoded = pd.get_dummies(vif_input, drop_first=True, dtype=float)
    vif_encoded = vif_encoded.replace([np.inf, -np.inf], np.nan)

    # Fill missing values for the diagnostic matrix without changing df_model.
    vif_encoded = vif_encoded.fillna(vif_encoded.median(numeric_only=True))

    # Remove constant columns because they do not help explain variation.
    vif_encoded = vif_encoded.loc[:, vif_encoded.nunique(dropna=False) > 1]
    return vif_encoded

def calculate_vif_table(vif_encoded):
    # Calculate VIF from an already encoded numeric dataframe.
    rows = []
    for index, feature_name in enumerate(vif_encoded.columns):
        try:
            vif_value = variance_inflation_factor(vif_encoded.values, index)
        except Exception:
            vif_value = np.inf
```

```

        rows.append({
            "encoded_vif_feature": feature_name,
            "vif": vif_value,
            "interpretation": interpret_vif(vif_value),
        })

        # Sort highest VIF first so the strongest multicollinearity signals appear
        ↪at the top.
        return pd.DataFrame(rows).sort_values("vif", ascending=False).
        ↪reset_index(drop=True)

```

The classification VIF check uses the same cleaned feature idea as the B2 classification model. The raw columns `person_income` and `loan_amnt` are excluded from the model/VIF inputs because `log_person_income` and `log_loan_amnt` keep the same information in a less skewed scale. `loan_grade_numeric` is excluded because `loan_grade` is already handled by ordinal encoding in B2. `default_on_file_binary` is excluded because `cb_person_default_on_file` is already encoded in the B2 pipeline.

```

[127]: classification_vif_features = [
        "person_age", "person_emp_length", "loan_percent_income", "loan_int_rate",
        "cb_person_cred_hist_length", "log_person_income", "log_loan_amnt",
        "income_to_loan_ratio", "employment_to_age_ratio",
        ↪"credit_history_to_age_ratio",
        "person_home_ownership", "loan_intent", "income_group",
        "loan_amount_group", "loan_percent_income_group", "age_group",
        "cb_person_default_on_file", "loan_grade",
    ]

    # Create the temporary encoded classification matrix used only for VIF.
    classification_vif_encoded = encode_features_for_vif(df_model,
        ↪classification_vif_features)

    # Show the encoded diagnostic columns before calculating VIF.
    classification_encoded_vif_features_table = pd.DataFrame({
        "encoded_vif_feature": classification_vif_encoded.columns
    })

    print("Total encoded classification VIF columns:",
        ↪len(classification_encoded_vif_features_table))
    classification_encoded_vif_features_table

```

Total encoded classification VIF columns: 37

```

[127]:
0          encoded_vif_feature
      person_age
1      person_emp_length
2      loan_percent_income
3          loan_int_rate

```

```

4          cb_person_cred_hist_length
5          log_person_income
6          log_loan_amnt
7          income_to_loan_ratio
8          employment_to_age_ratio
9          credit_history_to_age_ratio
10         person_home_ownership_OTHER
11         person_home_ownership_OWN
12         person_home_ownership_RENT
13         loan_intent_EDUCATION
14         loan_intent_HOMEIMPROVEMENT
15         loan_intent_MEDICAL
16         loan_intent_PERSONAL
17         loan_intent_VENTURE
18         income_group_Lower-Middle
19         income_group_Middle
20         income_group_Upper-Middle
21         income_group_High
22         loan_amount_group_Medium
23         loan_amount_group_Large
24         loan_amount_group_Very Large
25     loan_percent_income_group_Moderate Burden
26     loan_percent_income_group_High Burden
27     loan_percent_income_group_Very High Burden
28         age_group_Adult
29         age_group_Experienced Adult
30         cb_person_default_on_file_Y
31         loan_grade_B
32         loan_grade_C
33         loan_grade_D
34         loan_grade_E
35         loan_grade_F
36         loan_grade_G

```

```

[128]: # Calculate VIF from the same encoded classification matrix shown above.
classification_vif_table = calculate_vif_table(classification_vif_encoded)
classification_vif_table.head(15).style.format({"vif": "{:.2f}"})

```

```

[128]: <pandas.io.formats.style.Styler at 0x210c0b28080>

```

The regression VIF check uses the same cleaned feature idea as the B2 regression models. The raw `person_income` column is excluded from model/VIF inputs because `log_person_income` represents income on a more stable scale. `default_on_file_binary` is excluded because `cb_person_default_on_file` is already encoded in the B2 pipeline. These columns are removed only from modeling/VIF inputs, not from the saved dataset.

```
[129]: regression_vif_features = [
    "person_age", "person_emp_length", "cb_person_cred_hist_length",
    "log_person_income", "employment_to_age_ratio",
    ↪ "credit_history_to_age_ratio",
    "person_home_ownership", "loan_intent",
    "income_group", "age_group", "cb_person_default_on_file",
]

# Create the temporary encoded regression matrix used only for VIF.
regression_vif_encoded = encode_features_for_vif(df_model,
    ↪ regression_vif_features)

# Show the encoded diagnostic columns before calculating VIF.
regression_encoded_vif_features_table = pd.DataFrame({
    "encoded_vif_feature": regression_vif_encoded.columns
})

print("Total encoded regression VIF columns:",
    ↪ len(regression_encoded_vif_features_table))
regression_encoded_vif_features_table
```

Total encoded regression VIF columns: 21

```
[129]:      encoded_vif_feature
0      person_age
1      person_emp_length
2      cb_person_cred_hist_length
3      log_person_income
4      employment_to_age_ratio
5      credit_history_to_age_ratio
6      person_home_ownership_OTHER
7      person_home_ownership_OWN
8      person_home_ownership_RENT
9      loan_intent_EDUCATION
10     loan_intent_HOMEIMPROVEMENT
11     loan_intent_MEDICAL
12     loan_intent_PERSONAL
13     loan_intent_VENTURE
14     income_group_Lower-Middle
15     income_group_Middle
16     income_group_Upper-Middle
17     income_group_High
18     age_group_Adult
19     age_group_Experienced Adult
20     cb_person_default_on_file_Y
```

```
[130]: # Calculate VIF from the same encoded regression matrix shown above.
regression_vif_table = calculate_vif_table(regression_vif_encoded)
regression_vif_table.head(15).style.format({"vif": "{:.2f}"})
```

```
[130]: <pandas.io.formats.style.Styler at 0x210c0885f40>
```

### 1.1.5 VIF Interpretation and Feature Decision

The most direct duplicate predictors were already removed from the VIF and modeling feature lists. For example, `person_income` is not used together with `log_person_income`, `loan_amnt` is not used together with `log_loan_amnt` in classification, `loan_grade_numeric` is not used together with `loan_grade`, and `default_on_file_binary` is not used together with `cb_person_default_on_file`.

Even after this cleanup, some VIF values can still be high because a few kept features are naturally related by formula or business meaning. For example, `credit_history_to_age_ratio` is related to both `cb_person_cred_hist_length` and `person_age`. `employment_to_age_ratio` is related to both `person_emp_length` and `person_age`. `income_group` is related to `log_person_income` because both describe income level. Loan burden groups are related to `loan_percent_income` because both describe how large the loan is compared with income.

These remaining relationships do not mean the dataset is wrong. They show that credit-risk variables often describe connected borrower and loan behavior. The project keeps these features because ratio and grouped variables make the analysis easier to explain, and tree-based models such as Random Forest, Gradient Boosting, and CatBoost can usually handle related predictors better than coefficient-based models.

The VIF tables are used as diagnostic evidence only. A high VIF does not automatically mean a feature must be removed. It means the feature should be interpreted carefully because it overlaps with other predictors. High VIF can still appear for engineered features such as log transforms and ratios because they are intentionally created from original columns.

---

### 1.1.6 2.5 Save the Modeling Dataset

The feature-engineered dataset is saved as the Phase 2 modeling dataset. This file is used by the statistics and model-building notebook from Chapter 3 onward.

```
[131]: # Create the Phase 2 processed-data folder if it does not already exist.
Path(PHASE2_DATA_PATH).parent.mkdir(parents=True, exist_ok=True)

# Save the feature-engineered modeling dataset created through Chapter 2.
df_model.to_csv(PHASE2_DATA_PATH, index=False)

# Show a short confirmation so the saved dataset is easy to verify.
phase2_save_summary = pd.DataFrame([
    "saved_path": PHASE2_DATA_PATH,
    "rows": df_model.shape[0],
    "columns": df_model.shape[1],
```

```
}])  
phase2_save_summary
```

```
[131]:
```

|   |                                                  | saved_path | rows  | columns |
|---|--------------------------------------------------|------------|-------|---------|
| 0 | ../data/processed/phase2/credit_risk_cleaned.csv |            | 29567 | 23      |